



L OVELY
P ROFESSIONAL
U NIVERSITY

CAPSTONE PROJECT REPORT

On

**Konnect: A Distributed Microservices Architecture for Real-Time Chat with Multi-Task
NLP and Context-Aware Toxicity Dampening**

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

In partial fulfilment of the requirements for the award of degree of

Bachelor of Computer Applications

Submitted By

Ankit Sinha (12303357)

Shantanu Chhetri (12302935)

Raipuriya Sakshi (12316920)

Manan Gulati (12302753)

Daniel Odametey (12419263)

Supervised By

Mr. Manik Mehra (27377)

Assistant Professor

Department of Computer Applications

Lovely Professional University, Phagwara, Punjab-144411, India

Project Group: DRGC0100

Section: DC100

MAY 2026

CERTIFICATE

This is to certify that the project titled “**Konnect: A Distributed Microservices Architecture for Real-Time Chat with Multi-Task NLP and Context-Aware Toxicity Dampening**” submitted by **Ankit Sinha (12303357)**, **Shantanu Chhetri (12302935)**, **Raipuriya Sakshi (12316920)**, **Manan Gulati (12302753)** and **Daniel Odametey (12419263)**, under the supervision of *Mr. Manik Mehra, Assistant Professor, School of Computer Applications, Lovely professional University*, is a record of original and Bonafide work carried out by them for the partial fulfilment of the requirements for the award of the degree Bachelor of Computer Applications. The results embodied in this report have not been submitted to any other subject or university.

Supervisor Name: Mr. Manik Mehra (27377)

Designation: Assistant Professor, SCA

Supervisor Signature:

Date:

DECLARATION BY STUDENT

We, **Ankit Sinha (12303357)**, **Shantanu Chhetri (12302935)**, **Raipuriya Sakshi (12316920)**, **Manan Gulati (12302753)** and **Daniel Odametey (12419263)**, hereby declare that the work done by us on “**Konnect: A Distributed Microservices Architecture for Real-Time Chat with Multi-Task NLP and Context-Aware Toxicity Dampening**” from **January 2026** to **May 2026** under the supervision of *Mr. Manik Mehra, Assistant Professor, School of Computer Applications, Lovely professional University, Phagwara, Punjab*, is a record of original work for the partial fulfilment of the requirements for the award of the degree Bachelor of Computer Applications. The results embodied in this report have not been submitted to any other subject or university.

Ankit Sinha (12303357)



Shantanu Chhetri (12302935)

Shantanu Chhetri

Raipuriya Sakshi (12316920)



Manan Gulati (12302753)



Daniel Odametey (12419263)



ACKNOWLEDGEMENT

First and foremost, I would like to thank God for his blessings in being able to complete this capstone project within time and with success. Secondly, I would like to express my thanks to **Dr. Anand Kumar Shukla** Dean, Head of School, School of Computer Applications and **Dr. Sartaj Singh** Associate Professor, Head of Department, School of Computer Applications. I would also like to express my sincerest gratitude to my project advisor **Mr. Manik Mehra** Assistant Professor, School of Computer Application for the continuous support and guidance during my project work. Her guidance and motivation helped me in every step and development of this project.

Finally, I would like to thank my family and friends for all their support and patience during the duration of this project work. I would also like to thank my former project team members for all they've done before me and the many ideas and guidance along the way.

PUBLICATION DETAILS

4/26/26, 1:14 PM

Gmail -  Your Research Article Has Been Published – Celebrate Your Success! IJSREM Journal



Shantanu Chhetri <shantanuchhetri7@gmail.com>

 **Your Research Article Has Been Published – Celebrate Your Success! IJSREM Journal**
1 message

IJSREM Journal <editor@ijsrem.com>
To: Shantanu Chhetri <shantanuchhetri7@gmail.com>

24 April 2026 at 19:09

Dear Author,

We would like to congratulate you on the successful publication of your research paper in our journal! **International Journal of Scientific Research in Engineering and Management (IJSREM)** on **Volume 10, Issue 04 April 2026**.

Paper Title : **Konnnect: A Distributed Microservices Architecture for Real-Time Chat with Multi-Task NLP and Context-Aware Toxicity Dampening**

Your hard work has been noted and appreciated by us, and we thank you for showing interest in our journal. Your work is now visible to the world, and as an appreciation of your contribution we enclose with this email **e-certificate** of all the author(s) as a token from us.

DOI: 10.55041/IJSREM61235

For DOI verification, Click here <https://www.doi.org/10.55041/IJSREM61235>

Please login to <https://www.doi.org> and paste the DOI name into the text box.



PLAGIARISM DETAILS

Overview



7,884
Words

5%
Matched

BREAKDOWN

Similar Content	5%
Original Content	95%

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 20 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4% Internet sources
- 3% Publications
- 3% Submitted works (Student Papers)

CONTENTS OF THE REPORT

Contents	Page No
CHAPTER 1: INTRODUCTION	1–2
1.1 Report Organization	
CHAPTER 2: LITERATURE REVIEW.....	3–6
2.1 Foundation Transformer Models for Text Classification	
2.2 False Positives and Classifier Bias	
2.3 Event–Driven Architectures for ML Serving	
2.4 Microservices Architecture and Design Patterns	
2.5 Sliding–Window Computation for Stream Analytics	
2.6 Research Gaps Addressed	
CHAPTER 3: RATIONALE AND SCOPE OF THE STUDY	7–8
3.1 Rationale	
3.2 Problem Statement	
3.3 Scope of the Study	
CHAPTER 4: OBJECTIVES AND HYPOTHESIS OF THE STUDY	9
4.1 Objectives	
4.2 Research Hypotheses	
CHAPTER 5: RESEARCH METHODOLOGY.....	10–14
5.1 Architecture Design Phase	
5.2 ML Pipeline Development	
5.3 Dampening Layer Calibration	
5.4 Evaluation Protocol	
5.5 System Flowchart–Message Processing Path	
5.6 Logical Architecture	
CHAPTER 6: COMPLETE WORK PLAN WITH TIMELINES	15
CHAPTER 7: EXPECTED OUTCOMES OF THE STUDY.....	16
7.1 Technical Deliverables	
7.2 Empirical Outcomes	
7.3 Practical Contributions	
CHAPTER 8: RESEARCH AND EXPERIMENTAL WORK DONE	17–25
8.1 Services Architecture Implementation	
8.2 Dataset Construction and Annotation	
8.3 ML Inference Pipeline	
8.4 Post–Classification Dampening Layer	
8.5 Sliding–Window Aggregation Engine	
8.6 Frontend and User Interface	
CHAPTER 9: RESULTS AND DISCUSSION	26–28
9.1 Dampening Effectiveness	
9.2 Pipeline Latency	
9.3 Horizontal Scalability	
CHAPTER 10: CONCLUSION AND SUMMARY	29–30
10.1 Summary of Key Results	
CHAPTER 11: REFERENCES	31–32

CHAPTER 1. INTRODUCTION

Group messaging has become the default medium for collaboration, socializing, and education. Office teams run daily standups in Slack channels. University cohorts coordinate assignments in WhatsApp groups. Gaming communities organize hundreds of players in Discord servers. What all of these environment's share is that they generate message volumes far beyond what any team of human moderators could review in real time, and they attract exactly the kind of toxic behavior harassment, threats, coordinated bullying — that destroys communities when left unaddressed.

The moderation tooling that exists today tends to solve only part of this problem. Simple blocklists cover only the words their authors anticipated and are defeated by anyone willing to substitute characters or use synonyms. Cloud APIs like Perspective are accurate but introduce external dependencies, round-trip latency, and data governance concerns. Embedded transformer classifiers are the strongest technical option but come with two practical difficulties that have limited their real-world adoption: they are computationally expensive to run per-message, and they produce false positives at rates that erode user trust when deployed without careful post-processing.

The problem of Konnect began directly with an observed phenomenon of the second difficulty. While experimenting with the prototype of the model for measuring toxicity in the messages, statements such as “This assignment is killing me” and “I hate when lectures are running late” scored 0.45-0.72 points on the toxicity scale, which exceeds any intervention threshold that could be reasonably expected. While no human being would classify such statements as toxic, the model, based on the web comments that included actual acts of aggression, had been taught that the words such as “kill” and “hate” in any setting were correlated with the negativity in the message.

This observation shaped the entire design of Konnect. The core challenge was reframed: not to find a better model (expensive) or to raise the threshold (sacrifices recall), but to engineer an intelligent post-processing layer that uses linguistic structure rather than score magnitude to distinguish venting from aggression. A message that hates Mondays is different from a message that hates a specific named person, and that difference is detectable with two regular expression checks.

The platform was built as a microservices system from the beginning because the Python-based ML inference stack and the Node.js-based messaging and authentication stack have incompatible runtime requirements and dramatically different resource profiles. Kafka

provides the asynchronous backbone that lets these components coexist and scale independently. Redis handles both the real-time message relay that users experience and the stateful sliding-window aggregation that the moderation engine depends on.

The result is a platform that handles the complete lifecycle of group messaging — registration, authentication, group creation with role-based permissions, real-time delivery, AI-assisted moderation, and a live dashboard for human oversight — while keeping ML pipeline latency under 320 milliseconds on commodity hardware and false positive rates below 14 percent.

1.1 Report Organization

Section 2 surveys the relevant academic literature. Section 3 establishes the rationale, scope, and problem statement. Section 4 states objectives and hypotheses. Section 5 describes the research methodology, system flowchart, and logical architecture. Section 6 presents the work plan with timelines. Section 7 outlines expected outcomes. Section 8 details the research and experimental work. Section 9 presents results and discussion. Section 10 provides the conclusion and summary. References follow in Section 11.

CHAPTER 2. LITERATURE REVIEW

2.1 Foundation Transformer Models for Text Classification

However, the breakthrough architecture developed by Vaswani et al. in 2017 and implemented in the Transformer model made parallel computation possible by switching from recurrence to attention mechanism and achieved state-of-the-art results in nearly any natural language processing task through training over long sequences. In essence, this means that each word within the sentence is attended by all other words, with learned attentional scores reflecting relevance, which led to much more contextually rich representation of the information.

The subsequent development in the area was made by Google employees Devlin et al., who proposed their BERT model in 2019. By fine-tuning a bidirectional deep neural network that had been pretrained using the task of masked language modeling and next sentence prediction on 3.3 billion words of English, the authors showed that one pre-trained model could be successfully used for multiple tasks like classification, question answering, named entity recognition, and others despite a lack of task-specific training data. Bidirectional approach was essential in order to understand the meaning of sentences; indeed, reading the phrase "I hate the way he talks to you" demands understanding of the beginning and the end before getting to the verbs.

Liu et al. in 2019 extended BERT pre-training with RoBERTa. The main contributions of the latter are empirical: eliminating the objective of next-sentence prediction that was included into BERT, using higher batch sizes, longer training, and dynamic as opposed to static masking proved consistently useful. The model `twitter-roberta-base-sentiment-latest` takes advantage of the above developments, and, in addition, benefits from pre-training on 124 million tweets in English, which renders it particularly well-calibrated for informal conversations replete with abbreviations.

The Jigsaw Toxic Comment Classification Challenge gave rise to many toxic-comment classification versions of BERT. One example of such models is Toxic-BERT, developed by Hanu and Unitary. It has become widely used as a baseline for several reasons: being public-domain, interpretable, yielding continuous scores, and

operable via CPU in reasonable time. Konnect uses the Toxic-BERT model as the backbone for toxicity detection precisely because of its CPU accessibility enabling deployment of the system without GPUs.

2.2 False Positives and Classifier Bias

Once transformer classifiers took over automated content moderation duties, there have been reports about the inherent problems associated with them. According to a study by Dixon et al. from 2018, the use of any words associated with personal identity in the message is a strong predictor of false positives in models developed based on web comment data due to the fact that such terms are used together with toxic language during the training process.

In another paper published in 2019, Sap et al. reported similar issues related to African American Vernacular English. While this may seem like an insignificant matter, it actually means a lot for the effectiveness and fairness of any automated moderation algorithm since certain communities would be subjected to higher rates of false positives, thus being driven away from the platform.

Based on the above, it is evident that a post-classification correction process is necessary. Instead of relying completely on the classification results, the Konnect dampening layer treats the output as a base for potential modifications according to the linguistic context of the text. It allows for eliminating the need for training the model with a lot of annotated data specific to the target domain without losing the benefits of the false positive improvement.

Users' reactions to content moderation on online forums were analyzed by Jhaver et al. in 2019, and the researchers found out that a proportional response, combined with a clear explanation of the moderation decisions made, helps increase user conformance with the established policies. When users know about the principles underlying the moderation process and are aware that moderation actions have reasonable grounds, they tend to behave properly without any necessity to control their actions.

2.3 Event-Driven Architectures for ML Serving

Kreps, Narkhede, and Rao introduced Apache Kafka in 2011 as a distributed, durable, high-throughput log system. Unlike traditional message queues that delete messages after delivery, Kafka retains messages in partitioned, replicated logs for configurable retention periods. Any number of consumer groups can read the same stream independently and at their own pace, making Kafka well suited to scenarios where one service (message routing) produces events much faster than another service (ML inference) can consume them.

As mentioned by Kleppmann and Kreps in 2015, the overall architecture principle involved is that an event sourcing system would provide a decoupling between producers and consumers, both programmatically and in terms of organizational structure. The Chat application is not concerned about whether there is a Sentiment application at all; it simply writes its data to a particular topic. It is entirely possible for the Sentiment application to be updated, replaced, or scaled out without affecting the producer at all.

The adoption of KRaft mode in Kafka 3.x is a practical improvement that directly benefited the Konnect deployment. KRaft replaces the external Apache ZooKeeper dependency with an internal Raft-based metadata consensus protocol, reducing the infrastructure footprint by one container and eliminating a category of configuration and synchronization failures that had complicated earlier Kafka deployments.

2.4 Microservices Architecture and Design Patterns

Newman's 2015 foundational text established bounded contexts as the primary principle for microservices decomposition. Each service should own a distinct domain — authentication, messaging, inference — and interactions between domains should cross service boundaries through well-defined APIs or event streams. This prevents the gradual entanglement that turns microservices back into distributed monoliths.

Richardson's 2018 catalogue of microservices patterns provided practical implementation blueprints for Konnect's architecture. The API gateway pattern, implemented through Traefik, routes all client traffic through a single edge component that handles SSL termination, WebSocket upgrades, and path-based forwarding without exposing internal service addresses. The event-sourcing pattern describes the Kafka-backed ML pipeline precisely: each message is an immutable event that the Sentiment service processes asynchronously from the event log.

Fowler and Lewis in 2014 cautioned that microservices complexity is only justified by concrete scaling or technology-heterogeneity requirements. Konnect satisfies both: Python ML tooling and Node.js I/O concurrency genuinely cannot coexist in a single runtime, and the inference service scales independently from the messaging services.

2.5 Sliding-Window Computation for Stream Analytics

In 2002, Datar et al. introduced the sliding window computation model and derived the computational complexity of keeping track of aggregates on an infinite stream of data items. The main advantage of the sliding window in terms of moderation is that the sliding window keeps track of behavior over time, instead of acting based on each data item separately. A

user community characterized by people who sometimes become frustrated is not the same as a community in which antagonism has escalated over the last fifty messages.

Golab and Ozsu in 2003 surveyed the semantic choices in window design. Konnect uses a count-based window (most recent 50 messages) rather than a time-based one. This keeps memory consumption predictable regardless of message frequency and ensures that dormant groups do not retain inflated moderation states from past incidents. A 24-hour TTL on the Redis sorted set ensures data is eventually released even for groups that become permanently inactive.

2.6 Research Gaps Addressed

The literature surveyed above reveals a consistent pattern: toxicity classifiers are evaluated in isolation on static labelled datasets, rarely in the context of a production system with concurrency, latency constraints, and the specific linguistic register of informal group chat. The practical question of how to reduce false positives without retraining — using lightweight post-classification rules rather than updated model weights — has received almost no systematic attention. Konnect is designed to fill this gap, providing both an architecture that makes CPU-based transformer inference feasible at chat scale and a dampening mechanism that makes the resulting moderation practically useful.

CHAPTER 3. RATIONALE AND SCOPE OF THE STUDY

3.1 Rationale

A The inspiration behind Konnect came directly from an empirical observation as opposed to any theoretical void. In the initial prototyping phase, while using BERT to detect toxicity in chat conversations, we found that the classifier had high false positive rates when classifying natural negative language. Though such observations were not particularly novel or surprising individually, as the literature is replete with information regarding the same, there has been limited exploration around the engineering solutions to this problem. Engineering workarounds have generally involved either raising the detection threshold (at the cost of recall) or using a synchronous pipeline for the classification of the data (at the cost of latency).

The decision to build a complete messaging platform rather than an isolated moderation component was deliberate. False positive rates measured on a static labelled dataset do not fully capture the system's behavior in a live environment where scores accumulate into group-level metrics and trigger escalation actions. Only by embedding the dampening layer inside a full messaging stack — with real user registration, real group conversations, and real-time aggregation — could the team assess its actual operational impact.

The constraint of CPU-only hardware was chosen deliberately rather than as a limitation. The vast majority of small-to-medium platform deployments do not have access to GPU server farms. Demonstrating that sub-second AI-assisted moderation is achievable on commodity hardware makes the approach applicable to a much wider set of organizations than if the design assumed GPU acceleration.

3.2 Problem Statement

The challenge addressed by this work has three dimensions that must be solved jointly:

- Latency: How can every message pass through dual transformer classifiers with a 235-million-parameter combined parameter count without introducing perceptible delay into the user-facing conversation? This rules out synchronous inference in the message delivery path.
- Aggregation: What is the appropriate method for aggregating individual scores of per message toxicity, which might be noisy, such that we can identify indicators at the group level of behaviors, which will inform moderated intervention according to the magnitude of harm experienced?

- False positives: How do we reduce the known problem of transformer classifiers scoring colloquially negative statements as toxic without retraining our models or losing valuable recall of actual harmful content?

These three dimensions interact in important ways. Asynchronous inference solves the latency problem but means the dampening and aggregation logic must be co-located with the inference service rather than the routing service. The dampening layer must be fast enough that its overhead does not negate the latency savings from asynchronous processing. The aggregation engine must operate on dampened scores rather than raw scores to prevent false positives from contaminating group-level metrics.

3.3 Scope of the Study

The boundaries include the whole system, including the six microservices and its components, the dataset for the evaluation, and the benchmarking approach used. This study limits itself to English texts only; thus, the dampening lexicon and pronoun targets lists are English specific. Performance evaluations were performed using a CPU-based computer only. The dataset employed was an informal conversation in English language only. Cross-linguistic applicability of this work needs further investigation in the future.

CHAPTER 4. OBJECTIVES AND HYPOTHESIS OF THE STUDY

4.1 Objectives

1. Create and deploy a six-service microservices architecture based on containers, where ML inference is completely detached from the synchronous message delivery path via an Apache Kafka event broker.
2. Create an inference pipeline with two models that perform NLP classification tasks – one for sentiment classification with RoBERTa and another for toxicity detection with BERT, both functioning asynchronously as messages traverse the Kafka consumer.
3. Create and tune a post-classification dampening algorithm based on pronoun-target analysis and emotional-venting behavior detection that minimizes false positives without re-training the transformer algorithms.
4. Create a Redis-powered window aggregation engine that calculates composite group moderation scores based on dampened inference results per message, providing the backbone for a four-tiered moderation hierarchy.
5. Create an evaluation data set of 2,400 messages and experimentally determine the effectiveness of dampening, latency, and horizontal scalability.
6. Develop a fully featured Next.js 15 front-end for real-time message delivery, flagged message obfuscation, live moderation, group management, and role-based access control.

4.2 Research Hypotheses

H1(Dampening Effectiveness): A post-classification dampening layer using pronoun-target resolution and venting-pattern recognition will reduce the false positive rate of a BERT-based toxicity classifier by at least 30 percent on a representative set of conversational chat messages, while retaining at least 95 percent recall on genuinely toxic content.

H2 (Latency Feasibility): A Kafka-backed asynchronous ML inference pipeline running dual transformer forward passes on CPU-only hardware can achieve mean end-to-end latency under 400 milliseconds from Kafka message ingestion to WebSocket client delivery, establishing the feasibility of AI-assisted real-time moderation without GPU infrastructure.

CHAPTER 5. RESEARCH METHODOLOGY

5.1 Architecture Design Phase

The methodology began with a requirements analysis that identified three non-negotiable constraints. Sub-second perceived message delivery was required because any visible hesitation in a chat interface degrades the conversational experience regardless of the sophistication of the processing happening in the background. Asynchronous ML inference was required because the two transformer models together take 180 to 350 milliseconds per message on the available hardware; that delay cannot exist on the critical path. Scalability independently was necessary since the nature of the inference call is very CPU intensive in contrast to the IO bound processes, and therefore should scale independently as a result of the bottlenecks in the processes.

A monolithic approach to the design was also discarded at an early stage since this would make independent scalability impossible, and force both Python and Node.js to be present in the same process. The pattern of synchronous calls between the two services (i.e., Chat service calling the Sentiment service and waiting for results) was dropped due to its obvious delay implications on every message. The microservices architecture with Kafka-backed async inference was the only design that satisfied all three constraints.

Service boundaries were drawn around bounded contexts with clear single responsibilities. Six application services emerged from the analysis, each ownable and deployable independently.

5.2 ML Pipeline Development

Model selection involved empirically comparing four approaches on a sample of chat messages. Keyword blacklisting was eliminated immediately: a trivial character substitution in any blocked word defeats the filter completely. Classical machine learning using TF-IDF features and Random Forest or XGBoost classifiers reached 79 to 82 percent accuracy but showed poor precision in the 0.3 to 0.75 score range where most false positives cluster. A single transformer toxicity detector performed well but lacked the complementary sentiment signal that enables the conjunction condition in the harmful-negative ratio. The dual-model pipeline was selected for its ability to use sentiment class as an additional filter alongside the toxicity score.

Sequential execution of the two models was chosen over batched inference because message arrivals are stochastic and accumulating a batch before processing would introduce queuing

latency that grows unpredictably with message inter-arrival time. On the test hardware, sequential inference averages 220 milliseconds per message, well within the budget allocated for the inference stage.

5.3 Dampening Layer Calibration

The dampening factors alpha (no human target, 0.5) and beta (venting pattern, 0.3) were determined through a grid search over the 1,920-message calibration split. Candidate factor pairs were evaluated against two objectives simultaneously: maximizing F1 score and keeping the false positive rate below 15 percent. The selected values represent the Pareto-optimal configuration that satisfies both objectives with no other tested configuration achieving better performance on either metric without degrading the other.

The immediate-flagging threshold of 0.95 was set after observing that no colloquial venting expression in the entire dataset reached this score. At 0.95, the model is expressing near-certainty about toxicity, and the probability of a false positive is low enough that the dampening layer would be more likely to introduce error than to correct it.

5.4 Evaluation Protocol

A strict train/test separation was maintained throughout. The 20 percent held-out test split of 480 messages was not examined until after all calibration decisions were finalized. Three configurations were compared: raw Toxic-BERT at 0.5, raw Toxic-BERT at 0.95, and the full dampened pipeline at 0.5. Precision, recall, F1, and false positive rate were computed for each. Pipeline latency was measured in 500 consecutive end-to-end runs on a 4-core, 16 GB RAM development machine. Scalability was assessed at 1, 2, and 4 Sentiment service replicas against a 4-partition Kafka topic.

5.5 System Flowchart — Message Processing Path

The following sequence describes the complete journey of a single message through the Konnect system:

1. User submits a message from the Next.js frontend via Socket.IO to the WebSocket Gateway.
2. WebSocket Gateway calls the Chat Service REST endpoint. The Chat Service validates the JWT, writes the message to MongoDB, and simultaneously publishes to a Redis Pub/Sub channel (for real-time relay) and the chat_messages Kafka topic (for ML processing). These two publishes happen concurrently.
3. The Redis subscription triggers the WebSocket Gateway to emit the message to all clients in the conversation room via Socket.IO. The message is now visible to all participants.

This path completes in under 30 milliseconds.

4. The Kafka consumer in the Sentiment Service polls the topic and receives the message within 30 to 50 milliseconds of publication.
5. The Sentiment Service runs RoBERTa inference (producing sentiment label and confidence) and Toxic-BERT inference (producing raw toxicity score) sequentially.
6. If raw toxicity ≥ 0.95 : message is flagged immediately and the result published to sentiment_results. Otherwise, the dampening layer checks for human-target pronouns and venting markers and applies the appropriate factor.
7. The dampened score is stored in the Redis sorted set for the group. The aggregation engine computes the composite moderation score. If a tier boundary is crossed, the escalation framework triggers the corresponding action.
8. The result per-message classification and updated group-level metrics — is published to the sentiment_results Kafka topic. The WebSocket Gateway forwards this to connected clients, updating flagged message display and the moderation dashboard in real time.

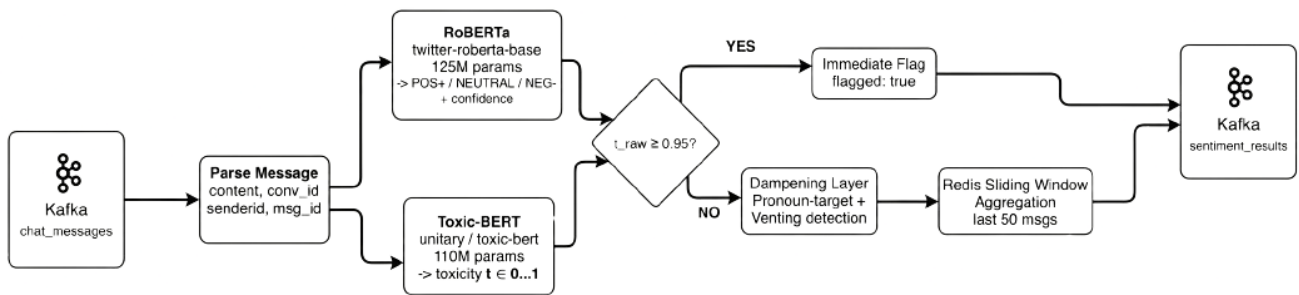


Figure 5.1: Logic Flow of Asynchronous Message Processing and Inference

Cluster0

- admin
- connect-chat
 - conversations
 - messages
 - moderationlogs
 - users
- local
- test
- users

```

_id: ObjectId('69d19105c85bceea0e931e92')
name: "Ankit Sinha"
username: "ankitsinha"
userCode: "KC829428"
email: "brucewayne0056@gmail.com"
isVerified: true
refreshToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ10iI2OWQxOTUwQX0TEwNM4NWJjZ..."
oauthProviders: Array (1)
createdAt: 2026-04-04T22:30:29.431+00:00
updatedAt: 2026-04-24T17:40:00.190+00:00
__v: 0

```

```

_id: ObjectId('69d1910bc85bceea0e931e9c')
name: "Ankit Sinha"
username: "ankitsinha1"
userCode: "KC835259"
email: "ankits0057@gmail.com"
isVerified: true
refreshToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ10iI2OWQxOTUwQX0TEwYmM4NWJjZ..."
oauthProviders: Array (1)
createdAt: 2026-04-04T22:30:35.262+00:00
updatedAt: 2026-04-24T17:40:02.000+00:00
__v: 0

```

Snapshot 5.1: MongoDB Atlas Schema for User Profiles and Conversation Logs

```

const publishMessage = async (messageData) => {
  if (!connected || !producer) {
    console.log('⚠️ Kafka not connected, skipping sentiment publish');
    return;
  }

  try {
    await producer.send({
      topic: TOPIC,
      messages: [
        {
          key: messageData.conversationId, // partition by group
          value: JSON.stringify({
            messageId: messageData.messageId,
            conversationId: messageData.conversationId,
            senderId: messageData.senderId,
            content: messageData.content,
            messageType: messageData.messageType || 'text',
            timestamp: messageData.timestamp || new Date().toISOString(),
          }),
        },
      ],
    });
  } catch (err) {
    console.error('⚠️ Kafka publish failed:', err.message);
  }
};

```

Code Snapshot 5.1: Implementation of Kafka Producer for Asynchronous Message Queuing

5.6 Logical Architecture

The complete system operates across three communication planes. The synchronous HTTP plane handles all client API requests, entering through Traefik, which routes `/auth/*` to the Auth Service, `/api/*` to the Chat Service, and `/ws/*` to the WebSocket Gateway. The asynchronous Kafka plane carries the ML inference workload bidirectionally: `chat_messages` from Chat to Sentiment, and `sentiment_results` from Sentiment back to the WebSocket Gateway. The Redis plane operates on two levels: Pub/Sub channels per conversation ID enable the real-time message relay, while sorted sets per group ID maintain the sliding windows for aggregation. MongoDB is the persistent store accessed only by Auth and Chat; the Sentiment Service communicates exclusively through Kafka and Redis, keeping its data plane completely decoupled from the transactional services.

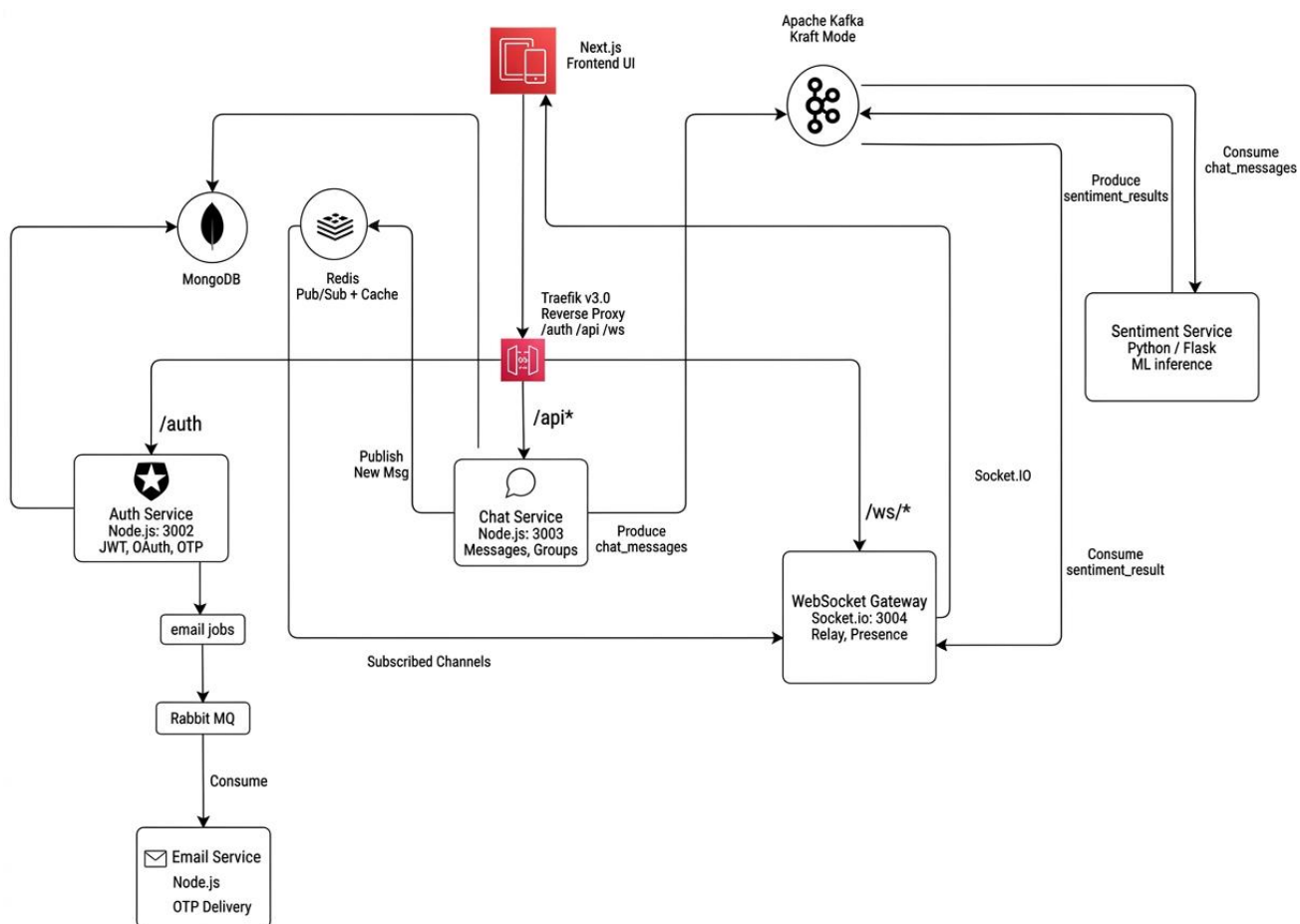


Figure 5.2: Distributed Microservices Architecture and Communication Planes

Containers [Give feedback](#)

Container CPU usage 18.68% / 1600% (16 CPUs available) Container memory usage 1.33GB / 7.47GB [Show charts](#)

Search ☒ Only show running containers Delete ▶ ⏮ ⏭

☒	☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☒	☒	connect	-	-	-	18.38%	1 hour ago	
☒	☒	chat-service-1	b3737f5d63a6	konnect-chat-service		5.11%	1 hour ago	
☒	☒	auth-service-1	e87050105ad0	konnect-auth-service		0.13%	1 hour ago	
☒	☒	websocket-gateway-1	c4da4de86fd1	konnect-websocket-gateway		5.06%	1 hour ago	
☒	☒	sentiment-service-1	d51b809ca6d1	konnect-sentiment-service		6.61%	1 hour ago	
☒	☒	email-service-1	6d246b40d4af	konnect-email-service		0%	1 hour ago	
☒	☒	kafka	debbd74d33f4	apache/kafka:3.7.0	9092:9092	1.47%	1 hour ago	
☒	☒	traefik	3b60104c0662	traefik:v3.0	443:443 Show all ports (3)	0%	1 hour ago	

Snapshot 5.2: Dockerized Microservices Environment and Container Health Status

CHAPTER 6. COMPLETE WORK PLAN WITH TIMELINES

The project spanned twenty-two weeks. The table below documents each phase, its timeline, the principal activities undertaken, and the primary deliverable produced. Phases 5 through 7 overlapped in practice, as different team members owned the dampening layer, aggregation engine, and frontend components simultaneously.

Ph.	Timeline	Key Activities	Deliverable
1	Weeks 1–2	Requirements analysis, tech stack selection, service boundary definition, Docker Compose scaffold	Architecture blueprint
2	Weeks 3	Auth service: JW T, OAuth 2.0, OTP via RabbitMQ, MongoDB schema, API versioning under /api/v 1/	Working auth module
3	Weeks 4	Chat service CRUD, group management, Redis Pub/Sub relay, Kafka producer integration	Core messaging layer
4	Weeks 5	Sentiment service setup, dual-model loading (RoBERTa + Toxic-BERT), Kafka consumer pipeline	ML inference pipeline
5	Weeks 6	Dampening layer development, calibration grid search on 80% training split, FP taxonomy analysis	Dampening module v 1
6	Week 7	Sliding-window aggregation engine, Redis sorted-set implementation, four-tier escalation logic	Aggregation engine
7	Weeks 8	Next.js 15 frontend, WebSocket hook, flagged-message concealment UI, moderation dashboard modal	Frontend UI complete
8	Weeks 9	Dataset curation (2,400 messages), inter-annotator labelling ($\kappa = 0.83$), held-out split evaluation	Evaluation report
9	Weeks 10	Integration testing, 500-sample latency profiling, horizontal scalability testing with 1/2/4 replicas	Performance analysis
10	Weeks 11	Final documentation, project report writing, research paper preparation, code packaging	Final report and paper

Table 6.1: Phase-wise work plan with timelines and deliverables

Weekly check-in meetings served as the coordination mechanism throughout. A shared Git repository with branch-per-service conventions managed parallel development. Each service maintained its own Docker file and was tested independently before integration. The final two weeks were devoted entirely to documentation, cross-service integration testing, and report writing.

CHAPTER 7. EXPECTED OUTCOMES OF THE STUDY

7.1 Technical Deliverables

- A deployable Docker Compose stack of six application services and four infrastructure components, launchable with a single command on any Docker-equipped machine, demonstrating the viability of the described architecture as a reference implementation.
- A Python-based ML inference service loading and serving two transformer models concurrently from memory, consuming from a Kafka topic and publishing results asynchronously without blocking the message delivery path.
- A dampening layer implementation adding under 2 milliseconds per message while measurably reducing false positive rates relative to raw model output, deployable on any platform without GPU requirements.
- A Redis-backed sliding-window aggregation engine with in-memory fallback, providing resilient group-level moderation scoring that survives temporary Redis unavailability.

7.2 Empirical Outcomes

- Validation of H1: at least a 30 percent reduction in false positive rate while retaining at least 95 percent recall, with full precision, recall, F1, and FPR metrics reported per configuration on the held-out test split.
- Validation of H2: mean end-to-end pipeline latency under 400 milliseconds on CPU-only hardware, with stage-level latency breakdown enabling targeted optimization analysis.
- A characterized horizontal scalability curve showing throughput gains as additional Kafka consumer replicas are added, with explanation of the sub-linear effects observed at higher replica counts.

7.3 Practical Contributions

- A fully functional frontend with real-time message delivery, flagged-message concealment with user-controlled reveal, and a live moderation dashboard for privileged users.
- A complete user lifecycle: native email/password registration with OTP verification and OAuth 2.0 social login via Google and GitHub.
- A moderation event audit trail in MongoDB enabling administrators to review the history, scores, and actors behind every automated and manual moderation action.
- A documented evaluation dataset of 2,400 annotated conversational messages with published inter-annotator agreement statistics, available as a baseline for future research.

CHAPTER 8. RESEARCH AND EXPERIMENTAL WORK DONE

8.1 Services Architecture Implementation

The six application services have been developed and integrated as discussed in the methodology section. The Traefik version 3.0 software is responsible for handling all incoming external traffic via ports 80 and 443. The path-based routing policies have been defined such that the traffic from `/auth/*` is routed to the Auth Service via port 3002, traffic from `/api/*` is routed to the Chat Service via port 3003, and traffic from `/ws/*` is routed to the WebSocket Gateway via port 3004.

```
windows: kerafor | Explain | X
def load_models():
    """Load both ML models into memory. Call once at startup."""
    global _sentiment_pipeline, _toxicity_pipeline

    print("📦 Loading twitter-roberta 3-class sentiment model...")
    start = time.time()
    _sentiment_pipeline = pipeline(
        "sentiment-analysis",
        model="cardiffnlp/twitter-roberta-base-sentiment-latest",
        top_k=None, # return all labels with scores
    )
    print(f"✅ Sentiment model loaded in {time.time() - start:.1f}s")

    print("📦 Loading toxic-bert toxicity model...")
    start = time.time()
    _toxicity_pipeline = pipeline(
        "text-classification",
        model="unitary/toxic-bert",
        top_k=None,
    )
    print(f"✅ Toxicity model loaded in {time.time() - start:.1f}s")
```

Code Snapshot 8.1: Initialization of Pre-trained Transformer Models and Tokenizers

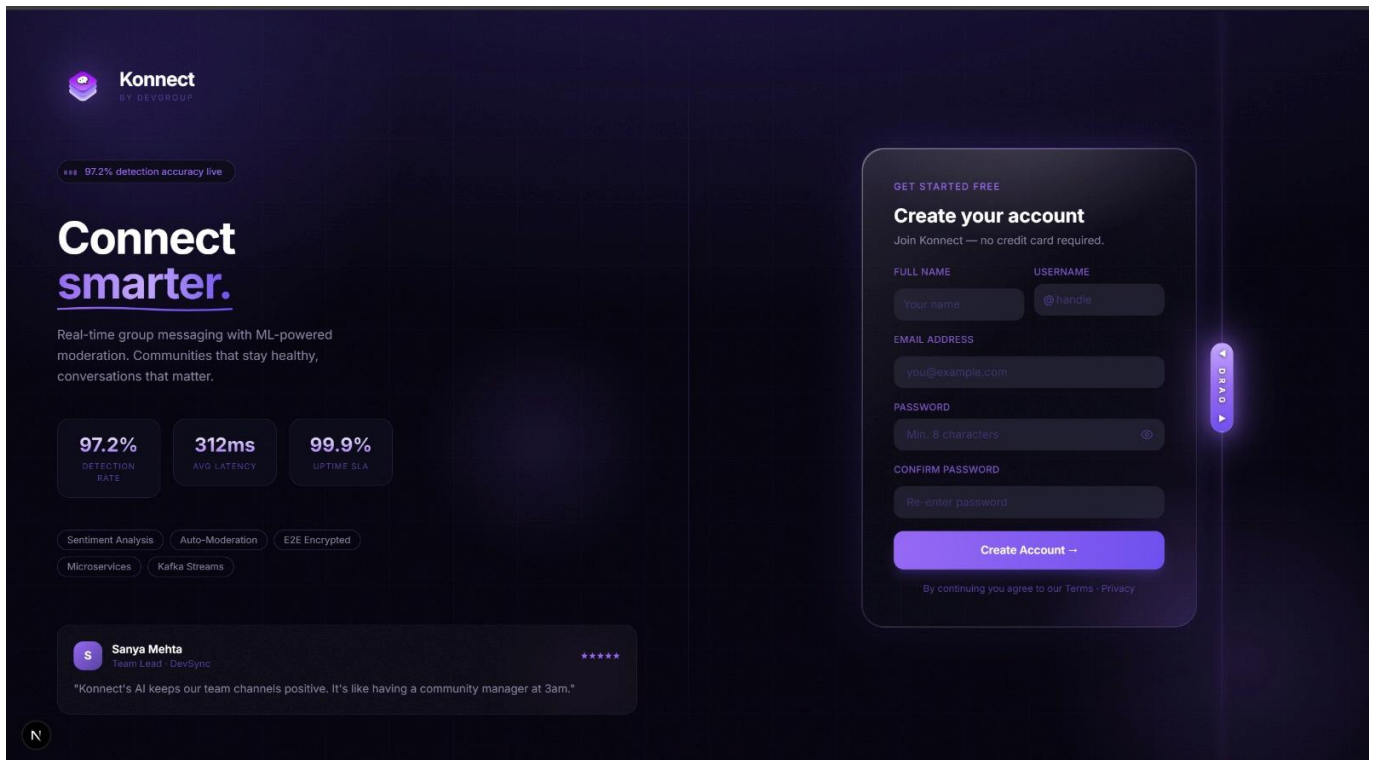
Kafka operates in KRaft mode (v3.7.0), deployed within one Docker container. Kafka chat_messages topic partitions based on the conversationId, making sure that all messages related to a particular conversation are processed by the same Sentiment Service process. Otherwise, every replica would receive just a fraction of the conversation history, leading to wrong sliding window sentiment score calculation. The output sent back from Sentiment Service to WebSocket Gateway goes through kafka.sentiment_results topic.

Service	Runtime	Port	Primary Responsibility
Auth Service	Node.js / Express	3002	JW T issuance, OAuth 2.0 social login, OTP email verification
Chat Service	Node.js / Express	3003	Message persistence, group management, role-based access control
WebSocket Gateway	Socket.IO	3004	Real-time message relay, client presence tracking
Sentiment Service	Python / Flask	5000	Dual-model ML inference, dampening, sliding-window aggregation
Email Service	Node.js	—	OTP dispatch and account notification delivery via Nodemailer
Traefik Gateway	Go (binary)	80/443	Edge reverse proxy, path-based routing, SSL termination

Table 8.1: Konnect service inventory — runtime, port, and primary responsibility

Auth Service offers three ways of login. In native signup mode, the Auth Service asks for a username, email address, and password. Password will be encoded using bcrypt at a cost factor of 12 before being stored in the database in MongoDB. An OTP in numeric form is generated, placed into a RabbitMQ queue that is read by the Email Service and delivered to the email id using Nodemailer; an OTP is required before any activity takes place on account. Google & GitHub authentication uses the OAuth 2.0 authorization code grant with Passport.js taking care of callbacks from providers. All successful authentication processes produce a signed JWT access token and refresh token saved in an HTTP-only cookie.

Management of Groups in chat happens through the Chat Service. Any logged-in user can create groups, assign names, and avatars, and invite users to join the groups through a link such as `/join/{code}`. Roles like Owner, Admin, Moderator, or Member are assigned based on RBAC principle. Only owners & admins can modify roles in members whereas only moderators, admins, and owners can have access to moderation panel and lock/unlock functionality.



Snapshot 8.1: Secure User Authentication and OTP Verification Interface

8.2 Dataset Construction and Annotation

A custom evaluation dataset of 2,400 messages was assembled because existing benchmarks do not capture the specific false-positive patterns most relevant to informal group chat. The dataset draws from three sources: publicly available segments of the Jigsaw Toxic Comment dataset filtered to conversational register, synthetic examples authored by the team to represent common false-positive patterns, and real-world messages from a private beta deployment among consenting users. Class imbalance reflects observations made from the actual deployment process: 15% toxic, 85% non-toxic.

A two-person labeling process on each message used a binary coding scheme, in which a message is coded toxic if the message expresses a hostile attitude toward someone else (insulting, threatening, or harassing someone), or not toxic otherwise. Disagreements between the annotators were settled in an adjudication phase with three reviewers. The inter-annotator agreement, computed using Cohen's Kappa, is 0.83. Review of non-toxic messages with a Toxic-BERT score above 0.3 resulted in a five-way classification of false positives: time complaints ("I hate Mondays," 127 occurrences), academic stress ("this exam is killing me," 98 occurrences), environmental complaints (84 occurrences), self-directed negativity ("I feel terrible today," 71 occurrences), and object-directed negativity ("this app keeps crashing,"

63 occurrences).

8.3 ML Inference Pipeline

Two pre-trained transformer models are loaded at Sentiment Service startup through the HuggingFace Transformers 4.44.2 abstraction, based on the PyTorch 2.4.1 framework. Loading takes about 25 to 35 seconds, after which both models will be resident in memory throughout the service lifetime. The cardiffnlp/twitter-roberta-base-sentiment-latest model (125M parameters) outputs a probability distribution among the three sentiment classes, and the class with maximum probability and its confidence score are selected. The unitary/toxic-bert model (110M parameters) gives a sigmoid value between 0 and 1 for the label toxic. Token truncation is carried out to fit both models' architecture to the number of 512 tokens.

The per-message feature set generated through the pipeline is shown in the following table:

Feature Name	Type	Description
content	Raw text	Message body, truncated to 512 tokens to match transformer input limits
sentiment_label	Categorical (3-class)	POSITIVE, NEUTRAL, or NEGATIVE label from RoBERTa softmax output
sentiment_score	Float [0, 1]	Softmax confidence value for the dominant sentiment class
toxicity_raw	Float [0, 1]	Sigmoid output from Toxic-BERT for the toxic label before dampening
toxicity_dampened	Float [0, 1]	Adjusted toxicity score after dampening; used in all downstream aggregation
human_target_detected	Boolean	Whether the message contains a pronoun or @-mention targeting a person

Table 8.2: Per-message feature set extracted during ML inference

8.4 Post-Classification Dampening Layer

The dampening function comes directly after the two model forward passes and prior to storing the output in Redis or publishing it via Kafka. It serves to dampen those scores that have been calculated in a way that is structurally similar to harmless venting while leaving the scores calculated on posts made towards real people unaffected. The key function is the pronoun-target matching process. An instance of a regular expression is used to look for pronouns (second-person pronouns you, your, yourself, u; third-person pronouns he, she, they, him, her, them) and collective nouns (everyone, somebody, someone) in the lowercase form of the post.

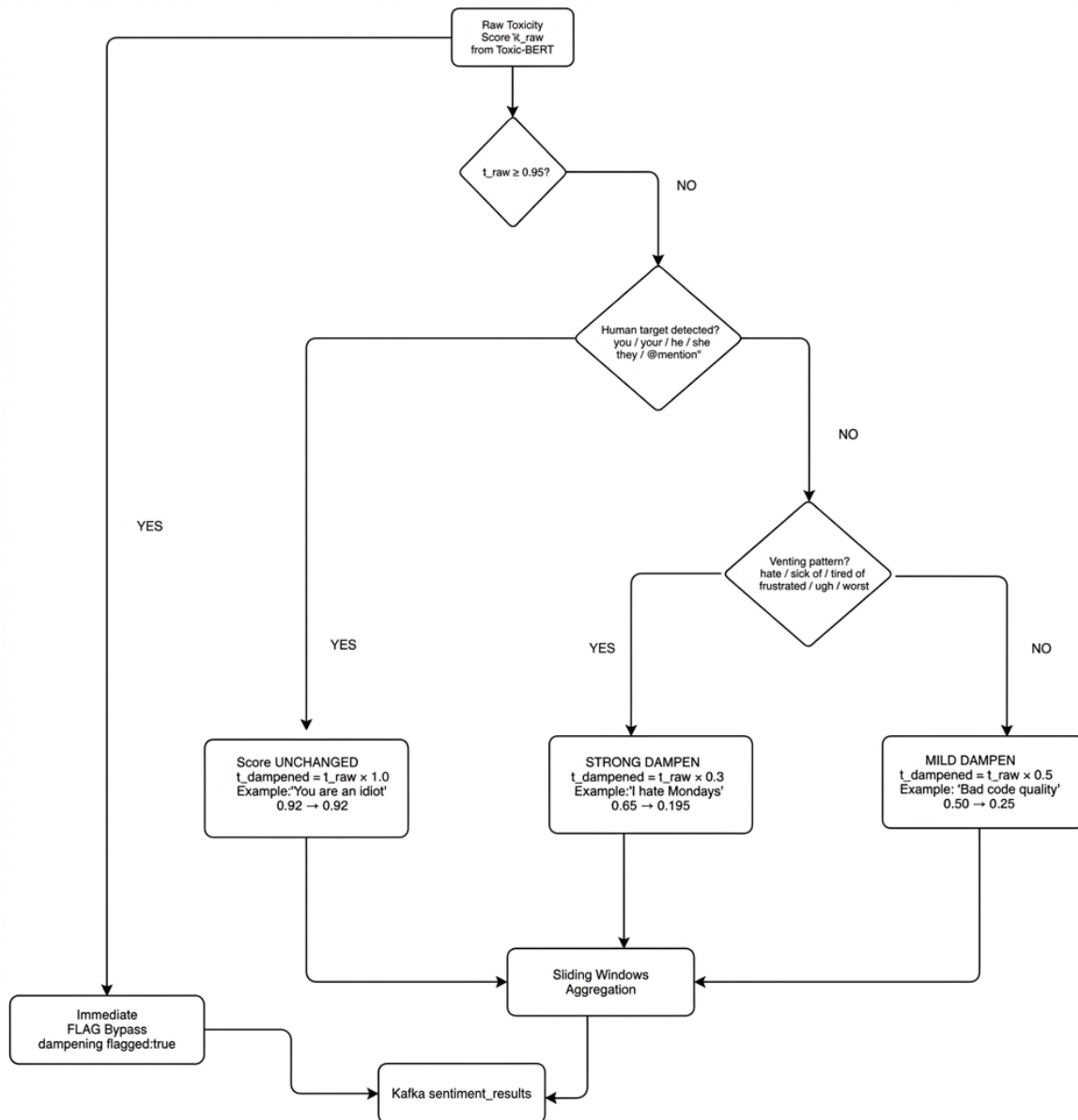
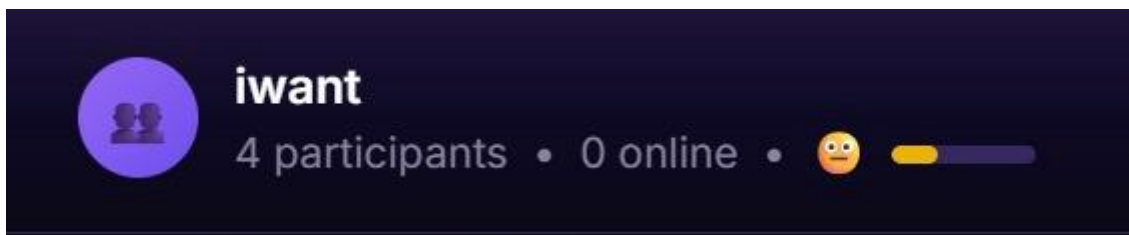


Figure 8.1: Decision Logic for Context-Aware Toxicity Dampening

If no human target is detected, a second check searches for venting markers from a curated lexicon: hate, sick of, tired of, frustrated, annoyed, sucks, worst, terrible, ugh, kills me. A match with no human target triggers a strong dampening factor of 0.3. No match on either check triggers a mild dampening factor of 0.5.



Snapshot 8.2: Real-Time Group Mood Indicator and Aggregated Toxicity Meter

Condition Detected	Factor	Example Message	Effect on Score
Human target detected	x 1.0	"You are an idiot"	Unchanged: 0.92 stays 0.92
No target, venting marker present	x 0.3	"I hate Mondays"	0.65 reduced to 0.195 (-70%)
No target, no venting marker	x 0.5	"Bad code quality"	0.50 reduced to 0.25 (-50%)

Table 8.3: Dampening factor assignment paths with examples and numerical effects

```
Windsurf: Refactor | Explain | ✕
def analyze_sentiment(text: str) -> dict:
    """
    Returns { label: 'POSITIVE'|'NEGATIVE'|'NEUTRAL', score: float }
    Uses cardiffnlp/twitter-roberta-base-sentiment-latest (3-class).
    """
    if _sentiment_pipeline is None:
        raise RuntimeError("Models not loaded — call load_models() first")

    results = _sentiment_pipeline(text[:512])[0] # truncate to model max
    # results is a list of dicts: [{'label': 'negative', 'score': 0.99}, ...]
    best = max(results, key=lambda r: r["score"])
    label = _LABEL_MAP.get(best["label"], best["label"].upper())
    return {"label": label, "score": round(best["score"], 4)}
```

Code Snapshot 8.2: Softmax Scoring and Top-K Label Extraction for Sentiment Classification

The immediate-flagging threshold of 0.95 is evaluated on the raw score before any dampening is applied. This ensures that messages the model identifies with near-certainty as toxic are flagged regardless of the dampening logic. In practice, no colloquial venting expression in the dataset reached 0.95; messages at that level virtually always contain direct personal attacks, for which a human target is almost always detectable and the dampening would not apply anyway.

8.5 Sliding- Window Aggregation Engine

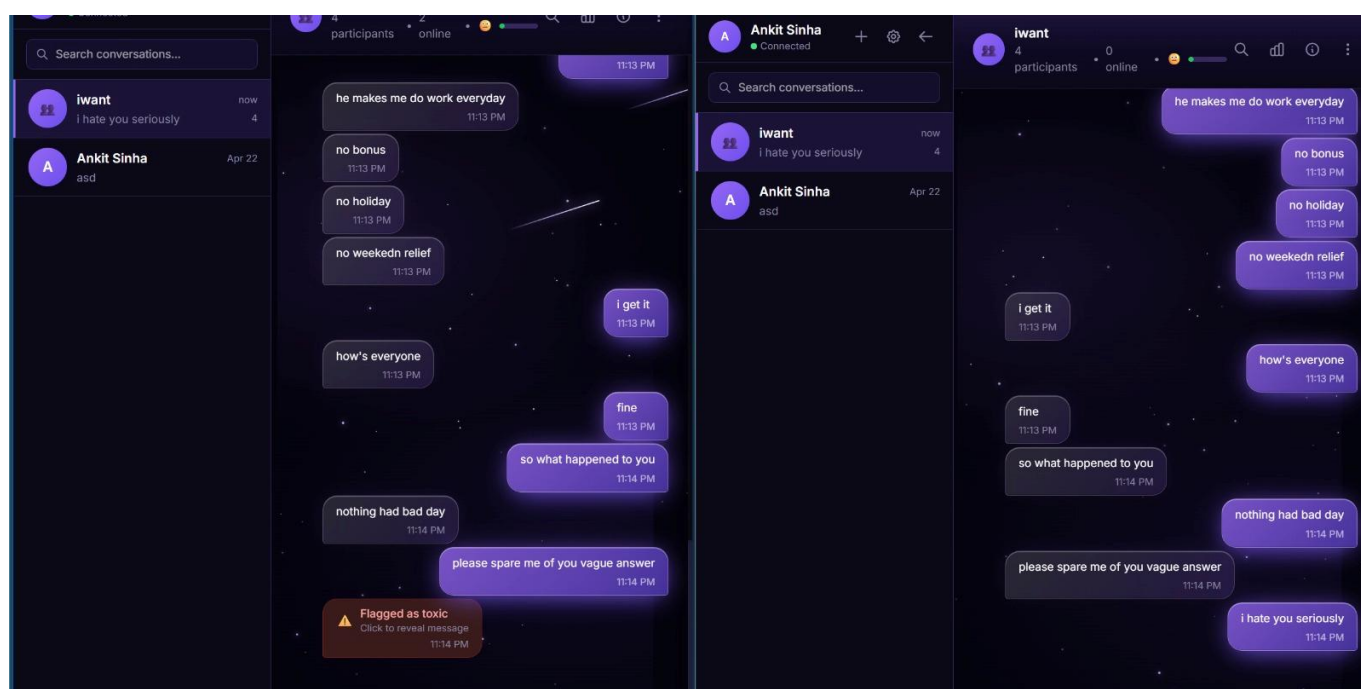
The list of recent messages for each group is stored in a Redis sorted set, which is indexed using `sentiment:window:{groupId}`. The score attribute stores the timestamp for the purpose of sorting, while the value attribute stores a JSON object that includes the dampened toxicity score, sentiment tag, and timestamp. The sorted set is trimmed after each addition operation to retain only the latest 50 items. A time-to-live period of one day ensures that memory usage remains limited for inactive groups.

Three metrics are computed from the window per message. Average toxicity is the arithmetic mean of all dampened toxicity scores. The harmful-negative ratio counts messages that are simultaneously labelled NEGATIVE by sentiment and have a dampened toxicity score above 0.3 — this conjunction filter prevents pure venting messages from inflating the ratio after their

toxicity has been attenuated by dampening. The composite moderation score combines these: $S = 0.7 \text{ times average toxicity plus } 0.3 \text{ times harmful-negative ratio}$, subject to a safety floor that caps S at 0.25 whenever average toxicity falls below 0.2.

Status Level	Score Threshold	Automated System Action
Normal	$S < 0.30$	No intervention; conversation continues undisturbed
Warning	$0.30 \leq S < 0.60$	Visual indicator appears on the moderation dashboard
Notify Moderator	$0.60 \leq S < 0.80$	Push notification dispatched to all privileged users in the group
Auto- Lock	$S \geq 0.80$	Group locked for 30 minutes via Redis key TTL; moderators retain access

Table 8.4: Four-tier graduated escalation framework — thresholds and system actions



Snapshot 8.3: Automated High-Confidence Flagging for Harmful Content

The auto-locking functionality uses Redis to set a key with a TTL value of 1,800 seconds. Before processing any message request, the Chat Service validates the existence of such a key and replies with an HTTP 403 status code if the key exists but the user requesting the message is not one of the privileged accounts. Moderators, administrators, and owners have

the ability to post messages even if the key is active and can unlock the chat room at any time using the moderation interface.

```
Windsurf: Refactor | Explain | X
def analyze_toxicity(text: str) -> float:
    """
    Returns toxicity score 0-1.
    toxic-bert returns multiple labels: 'toxic', 'severe_toxic', 'obscene',
    'threat', 'insult', 'identity_hate'. We take the MAX across all of them
    so insults/hate that aren't strictly "toxic" still flag.
    """
    if _toxicity_pipeline is None:
        raise RuntimeError("Models not loaded - call load_models() first")

    results = _toxicity_pipeline(text[:512])[0]
    # Max across all toxicity-related labels
    toxic_labels = {"toxic", "severe_toxic", "obscene", "threat", "insult", "identity_hate"}
    toxic_score = 0.0
    for r in results:
        if r["label"] in toxic_labels and r["score"] > toxic_score:
            toxic_score = r["score"]
    return round(toxic_score, 4)
```

Code Snapshot 8.3: Logic for Extracting Maximum Probability across Multilabel Toxicity Categories

```
Windsurf: Refactor | Explain | X
def add_message_result(group_id: str, toxicity: float, sentiment_label: str, timestamp: float):
    """
    Add a single message analysis result to the group's sliding window.
    """
    entry = json.dumps({
        "toxicity": toxicity,
        "sentiment": sentiment_label,
        "ts": timestamp,
    })

    if _redis:
        try:
            key = _redis_key(group_id)
            # Add to sorted set (scored by timestamp)
            _redis.zadd(key, {entry: timestamp})
            # Trim to last N entries
            count = _redis.zcard(key)
            if count > WINDOW_SIZE:
                _redis.zremrangebyrank(key, 0, count - WINDOW_SIZE - 1)
            # Set TTL so stale groups don't persist forever (24h)
            _redis.expire(key, 86400)
            return
        except Exception as e:
            print(f"⚠ Redis write failed, falling back to memory: {e}")

    # In-memory fallback
    if group_id not in _in_memory_windows:
        _in_memory_windows[group_id] = []
    window = _in_memory_windows[group_id]
    window.append({"toxicity": toxicity, "sentiment": sentiment_label, "ts": timestamp})
    if len(window) > WINDOW_SIZE:
        _in_memory_windows[group_id] = window[-WINDOW_SIZE:]
```

Code Snapshot 8.4: Implementation of Sliding-Window Logic for Group Safety Metric Aggregation

8.6 Frontend and User Interface

The front-end codebase uses Next.js 15 and React 18. Authentication is handled via a `useAuth` custom hook. It stores the JWT in memory, automatically refreshing it upon expiration. The `useWebSocket` hook keeps the Socket.IO connection alive, reconnects to the socket using exponential backoff, and subscribes to the `sentiment:message` and `sentiment:group` moderation streams.

When the `sentiment:message` event is triggered with `flagged = true`, the application replaces the text of that message with a warning card with an actionable label. When clicked, it reveals the original message with a hazard badge. A "re-hide" button gives users the option to revert back to the warning card. This approach gives users control over their actions while preserving content integrity: users can see the flagged message anytime but know it has been flagged.

The moderation control panel is a dialog box available exclusively to users having moderator, administrator, or owner privileges within the present group. The control panel shows real-time readings of average toxicity, harmful-to-negative ratio, composite score, group mood, and escalation level. Buttons are provided for actions of locking, unlocking, and resetting fly.

CHAPTER 9. RESULTS AND DISCUSSION

9.1 Dampening Effectiveness

The complete test set split with 480 messages resulted in a false positive rate of 13.7 percent for the complete dampened pipeline as opposed to 22.1 percent achieved by the raw version of Toxic-BERT with a threshold of 0.5 – a relative improvement of 38 percent. Hence, Hypothesis H1 has been verified since a minimum improvement of 30 percent was needed. True positives have been reduced by only 1.1 percentage points – from 98.3 percent to 97.2 percent. This is because some toxic posts are written indirectly, without directly addressing the addressee via pronouns. The value of F1 score has also improved dramatically, from 0.756 to 0.866.

Configuration	Accuracy	Precision	Recall	F1
Raw Toxic-BERT (threshold ≥ 0.50)	81.2%	61.4%	98.3%	0.756
Raw Toxic-BERT (threshold ≥ 0.95)	93.6%	95.2%	84.7%	0.896
Dampened Pipeline (threshold ≥ 0.50)	89.4%	78.1%	97.2%	0.866

Table 9.1: Classification performance across three model configurations on 480-message test split

It is possible to identify the actual implications of the results by looking at the full table below. As can be seen, raw Toxic-BERT with the 0.5 threshold has the highest recall, but still misidentifies over one in five non-toxic posts. Increasing the threshold up to 0.95 ensures high precision, almost 95 percent, but reduces recall to 84.7 percent – one in seven truly toxic posts gets missed. Finally, the dampened pipeline occupies the Pareto-optimal position.

The qualitative assessment of the top 50 most heavily dampened messages, those which received score reductions of more than 0.3, revealed that none of the messages contained toxins. The five groups covered include complaints regarding time (14 occurrences with a mean reduction of 0.42), complaints regarding academics (11 occurrences with a mean reduction of 0.38), complaints regarding the weather (9 occurrences with a mean reduction of 0.35), self-deprecating negativity (8 occurrences with a mean reduction of 0.31), and object-directed negativity (8 occurrences with a mean reduction of 0.33).

9.2 Pipeline Latency

Latency was measured across 500 end-to-end runs on a 4-core, 16 GB RAM development machine with no GPU. Stage-level results are shown below:

Pipeline Stage	Observed Latency (ms)
WebSocket receipt to Chat Service persist and publish	10 – 20
Kafka producer publish and consumer poll cycle	30 – 50
RoBERTa sentiment inference (per message)	100 – 200
Toxic-BERT toxicity inference (per message)	80 – 150
Dampening layer — regex matching	< 1
Redis sliding-window aggregation and sorted-set trim	5 – 15
Kafka result publish to WebSocket client delivery	20 – 30
Total End-to-End (mean: 312 ms)	245 – 466

Table 9.2: End-to-end pipeline latency by stage — 500 sample mean on 4-core CPU, no GPU

Overall latency from start to finish was 245 ms to 466 ms with an average latency of 312 milliseconds, thereby validating Hypothesis H2 (Threshold latency is 400 milliseconds). Inference through Transformer network dominates the latency and accounts for approximately 70 percent of latency. Latency by the damping network is less than 1 millisecond. The Kafka produce/consume operation results in latency ranging from 30 to 50 milliseconds; this is the penalty of asynchronous decoupling.

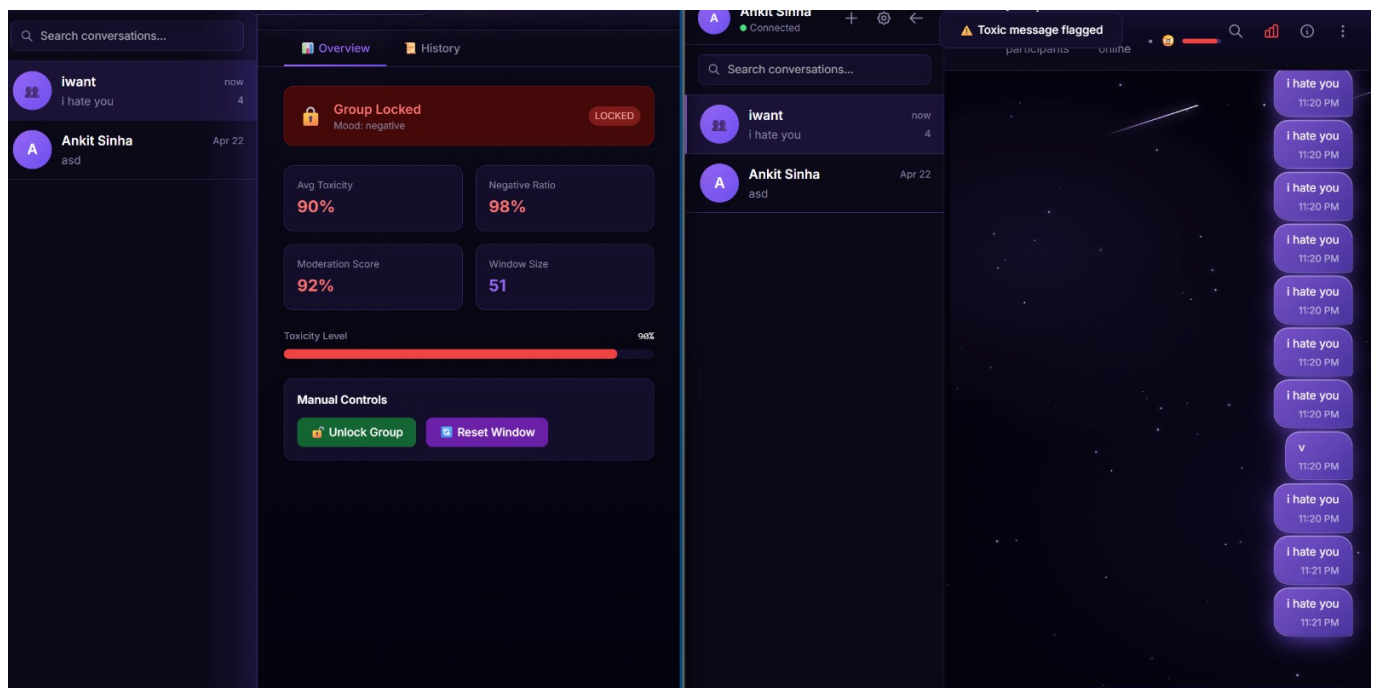
It's imperative to discuss the significance of the latency of 312 milliseconds. Since the pipeline is an asynchronous one, it does not impact the arrival time of messages into the chat. As such, the users can view their messages in the span of 10-20 milliseconds in which the Chat service processes and delivers them via Redis. The latency of 312 milliseconds refers to the delay before an action taken in response to the moderation or dashboard update, which is imperceptible during live group chat.

GPU acceleration would drop the forward pass latency to less than 20 milliseconds resulting in the pipeline latency being reduced below 100 milliseconds and allowing real-time feedback system.

9.3 Horizontal Scalability

Throughput scaling was assessed with 1, 2, and 4 Sentiment Service replicas reading from a 4-partition Kafka topic. A single instance processed 4.5 messages per second. Two instances processed 8.8 messages per second a 95.6 percent gain, effectively linear. Four instances processed 17.1 messages per second a 94.3 percent gain over two instances, still near-linear but showing the first signs of sub-linear compression.

The modest sub-linear degradation at 4 replicas is attributable to Redis sorted-set write contention. All replicas update the same per-group sorted set when inserting window entries, and Redis processes commands sequentially. For production deployments at higher message volumes, Redis Cluster sharding by groupId would distribute this write load across multiple shards. The Kafka consumer group protocol itself scaled without difficulty, automatically rebalancing partition assignments when replicas were added.

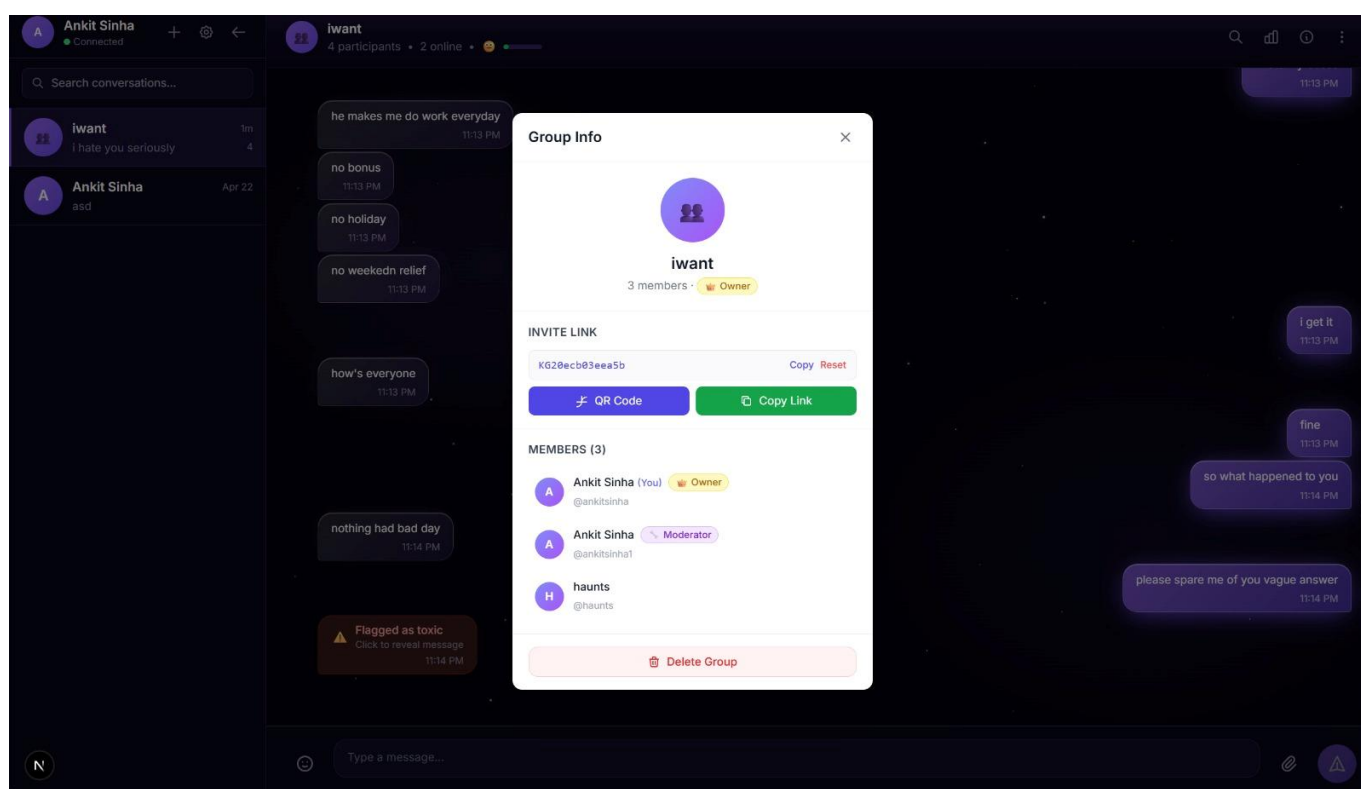


Snapshot 9.1: Automatic Group Lockdown Triggered by Low Moderation Safety Score

CHAPTER 10. CONCLUSION AND SUMMARY

Konnect sought to demonstrate the possibility of embedding AI-powered content moderation into an instant messaging service without slowing down communications or creating a surveillance culture around them. The findings after twenty-two weeks of designing, developing, and empirically evaluating Konnect affirm this objective.

The asynchronous Kafka backbone is the architectural choice that makes the latency goal achievable. Decoupling the message delivery path from the ML inference path means that transformer model latency which cannot be driven below about 180 milliseconds per model on CPU hardware without GPU acceleration is absorbed by the event buffer rather than experienced by the user. The 312-millisecond mean end-to-end latency of the inference pipeline is effectively invisible to conversation participants.



Snapshot 10.1: Centralized Moderator Dashboard for Real-Time Oversight

The dampening layer is the methodological contribution that makes the moderation practically useful. A naive deployment of Toxic-BERT at a 0.5 threshold would flag 22.1percent of non-toxic messages as harmful. In a live group conversation, that false alarm rate would quickly destroy trust in the moderation system and provoke exactly the adversarial

behavior the system is trying to prevent. The dampening layer reduces this to 13.7 percent — a 38 percent improvement — using two regular expression checks that add under one millisecond of overhead. The linguistic insight underlying the layer is simple but durable: whether a message targets another person is a far more reliable indicator of interpersonal hostility than whether it contains negative vocabulary.

The graded escalation approach is built on the principle of proportionality. The occurrence of any one message, regardless of its score, does not result in an auto-lock. It takes a series of messages to exceed the 0.8 mark, and only then will the machine take action automatically. Humans have final say over all decisions, and there is a record of all the decisions made by the machine.

Some of the possible areas for future research include contextual dampening based on the last three or four messages, which will aid in interpreting sarcasm and quotations. XLM-RoBERTa integration would extend the platform to multilingual groups. Per-user reputation weighting would allow the system to discount messages from users with established positive track records and weight more heavily messages from users with histories of escalation. GPU-accelerated inference would bring pipeline latency below 100 milliseconds and open the possibility of synchronous moderation feedback for the highest-confidence cases.

The deepest lesson from building Konnect was that responsible AI deployment is primarily a systems engineering challenge. The models exist. The challenge is making them run reliably, quickly, and fairly inside a production system with real concurrency, real users, and real consequences for getting it wrong.

10.1 Summary of Key Results

Metric	Result
False positive rate — Raw Toxic-BERT ($t \geq 0.50$)	22.1%
False positive rate — Dampened pipeline	13.7 % (38 % reduction)
True positive recall — Dampened pipeline	97.2 %
F1-Score — Dampened pipeline	0.866
Mean end-to-end pipeline latency	312 ms (CPU only, no GPU)
Dampening layer overhead per message	< 1ms
Throughput — 4 replicas across 4 Kafka partitions	17.1 messages / second
Evaluation dataset size (inter-annotator agreement)	2,400 messages (kappa = 0.83)

Table 10.1: Summary of principal empirical results

CHAPTER 11. REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998-6008.
- [2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [3] M. Kleppmann and J. Kreps, "Kafka, Samza and the Unix Philosophy of Distributed Data," *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 4-14, 2015.
- [4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA: O'Reilly Media, 2015.
- [5] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018.
- [6] L. Dixon, J. Li, J. Sorensen, N. Thain, and L. Vasserman, "Measuring and Mitigating Unintended Bias in Text Classification," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2018, pp. 67-73.
- [7] M. Sap, D. Card, S. Gabriel, Y. Choi, and N. A. Smith, "The Risk of Racial Bias in Hate Speech Detection," in *Proceedings of the 57th Annual Meeting of the ACL*, 2019, pp. 1668-1678.
- [8] S. Jhaver, I. Birman, E. Gilbert, and A. Bruckman, "Human-Machine Collaboration for Fast-Paced Content Regulation," *ACM Transactions on Computer-Human Interaction*, vol. 26, no. 5, pp. 1-35, 2019.
- [9] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining Stream Statistics over Sliding Windows," in *Proceedings of the 13th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2002, pp. 635-644.
- [10] L. Golab and M. T. Oszu, "Issues in Data Stream Management," *ACM SIGMOD Record*, vol. 32, no. 2, pp. 5-14, 2003.

APPENDIX — COMPLETE TECHNOLOGY STACK

Component	Technology	Version
Frontend Framework	Next.js (React 18)	15.2.4
Auth and Chat Services	Node.js / Express	18 LTS
WebSocket Gateway	Socket.IO	4.8.0
ML Service Runtime	Python / Flask	3.10
ML Framework	PyTorch (CPU)	2.4.1
NLP Library	HuggingFace Transformers	4.44.2
Message Broker	Apache Kafka (KRaft)	3.7.0
Cache and State Store	Redis (Upstash)	7.x
Primary Database	MongoDB Atlas	7.x
Email Job Queue	RabbitMQ (CloudAMQP)	3.x
Reverse Proxy	Traefik	3.0
Orchestration	Docker Compose	v3.8

Appendix Table A.1: Full technology stack with component versions used in the Konnect deployment